LiveRecall: Networking Design and Evaluation

1. Architecture and Protocol

1.1 System Architecture

The supplied caption_source.py emits full-text revisions over WebSocket. One asyncio server routes hello messages. **CaptionState** retains active/latest revisions and finalized IDs; **History** retains recent finals. Each client owns a **FinalAwareQueue**, sender and supervised session. **NORMAL** uses FIFO; **DEGRADED** separates a final/control FIFO and latest-active map, with a final-aware scheduler. **RecoveryPlan** supplies a bounded prefix before live delivery. The browser maintains display/recall state; lecture state is lost on server restart.

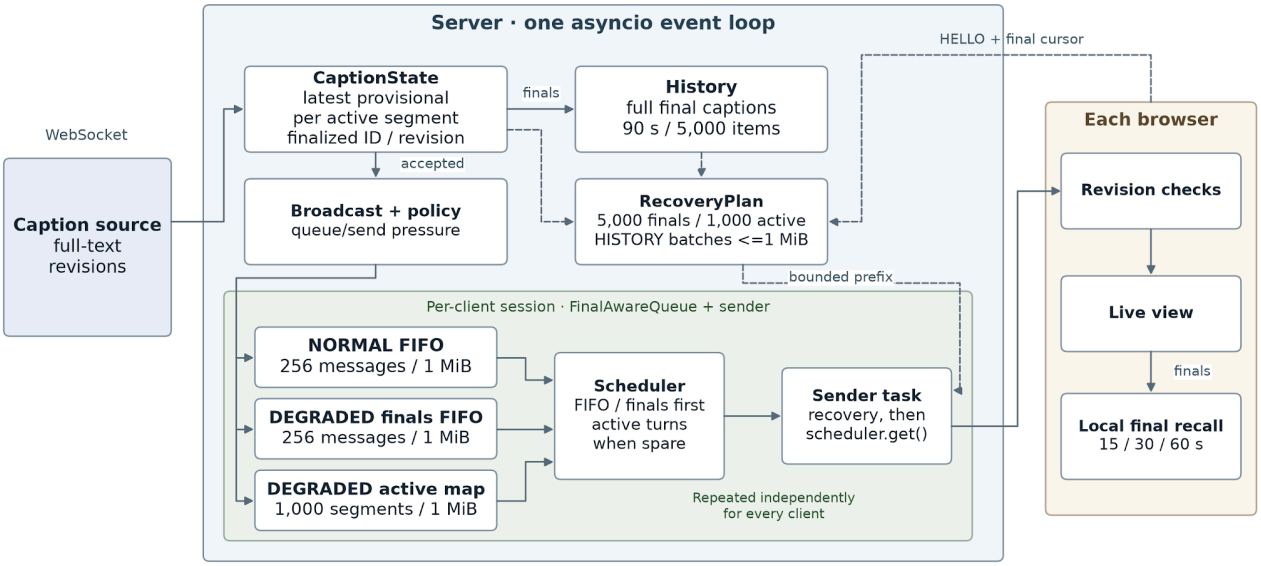


*Figure 1. Each client uses **NORMAL** FIFO or **DEGRADED** final/control FIFO plus latest-active map. A final-aware scheduler feeds the same sender; **RecoveryPlan** supplies its initial bounded prefix. Modes are exclusive.*

1.2 Application Protocol

Messages are JSON text frames. SOURCE_HELLO identifies a source; CLIENT_HELLO supplies client_id, session, the last accepted final cursor and requested history window. CAPTION_UPDATE carries session, segment, revision, source_ts, final and full revision text. HISTORY contains oldest-first final items and may span multiple bounded batches following CLIENT_HELLO. Batches precede active snapshots and live updates. SOURCE_DONE is diagnostic; hello/session fields do not provide authentication or independent multi-lecture state.

2. Revision and Recovery Semantics

2.1 Revision State and Bounded History

The server accepts an active segment only at a strictly newer revision and rejects all changes after finalization. The browser independently applies equivalent revision/final rules. Server state caps active segments at 1,000 and remembered final revisions at 10,000; a segment watermark rejects forgotten ranges. Full finals are retained separately for 90 seconds and at most 5,000 items. Retention uses server monotonic acceptance time, with pruning on insertion/query. Count limits can shorten retained context; batching avoids truncation at a total message-size budget.

2.2 Local Recall and Browser Behavior

Live and recovered captions share applyCaption()/acceptUpdate(), so accepted revisions replace one segment line rather than adding duplicates. Provisional and final styling differs. The browser tracks at most 2,000 segments and renders at most 300 lines, with follow-live scrolling and a jump control. Accepted finals enter a local rolling recall store, filtered by source timestamps: default 30 seconds, selectable 15/30/60 seconds. Normal recall reads this store without a server request. This timestamp comparison assumes synchronized source/browser clocks.

2.3 Reconnect and Late-Join Recovery

When a client reconnects: (1) it sends the last final it holds; (2) the server checks the request and, without pausing, takes a snapshot of the missed finals and current drafts and registers the client; (3) it sends the missed finals in HISTORY batches of at most 1 MiB, then the current drafts, then live updates. Only one batch is serialized at a time. Oversized individual items fail before partial delivery; expired/count-pruned history, live overflow and disconnects still limit recovery.

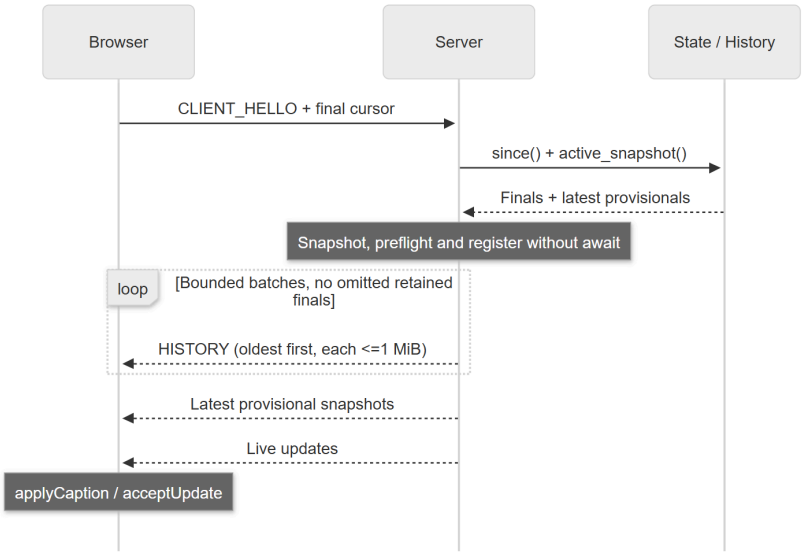


Figure 2. Snapshot and registration do not yield; the sender streams HISTORY batches, active state, then live updates.

3. Per-Client Buffering and Overload Handling

3.1 Queues, Senders and Session Lifecycle

Each client owns one sender/session. **NORMAL** uses a 256-message/1 MiB FIFO. **DEGRADED** uses a final/control FIFO with those caps and a latest-active map capped at 1,000 segments/1 MiB. Modes are exclusive: at most 1,256 pending live items/2 MiB, excluding Python/socket overhead, one in-flight item and bounded recovery references. Producers never await client network I/O. Teardown unregisters, clears state and cancels/awaits tasks; send/close deadlines remain 10 s/3 s.

3.2 Monitoring and Overload Policy

Monitoring separates queued state, in-flight sends, and transport-buffer bytes. Enter **DEGRADED** at 50% of the **NORMAL** count/byte budget or 0.5 s queue/send age; exit at ≤20% and ≤0.15 s. The asymmetric thresholds provide hysteresis: degradation begins only after pressure becomes material, while recovery requires the queue and sender to become substantially healthier before NORMAL delivery resumes. This reduces mode oscillation and prevents a recovering client from immediately rebuilding the same backlog. The chosen thresholds are engineering parameters rather than analytically optimal values; Section 6 therefore treats further threshold tuning as future work. Finals retain FIFO order; active entries keep their latest same-segment revision and are removed by newer finals. Below four pending finals, four protected sends allow an active turn; at backlog ≥min(4, FIFO capacity), drain finals first to avoid a fixed 80% throughput cap. Return to **NORMAL** restores retained arrival order when it fits. Finals and provisionals have separate capacity. Hard lane exhaustion can still cause **1013** eviction; disconnects clear pending data. Keepalive 2 s/5 s is independent of displayed-caption lag.

The overload policy is intentionally semantic rather than purely FIFO. Under healthy conditions, preserving every revision maximizes Normal Update Coverage; under sustained pressure, however, transmitting superseded provisional revisions consumes capacity without improving the client's eventual caption state. DEGRADED therefore prioritizes finalized captions and the latest useful active revision while bounding obsolete intermediate state. The policy is per-client so that congestion at one receiver does not alter delivery behavior for clients that remain healthy. Separate entry and exit thresholds provide hysteresis, avoiding repeated mode changes around a single pressure threshold.

4. Experimental Evaluation

4.1 Experimental Setup and Metrics

- Host: GB10; Ubuntu 24.04.4 LTS, ARM64, 20 logical CPUs.
- Runtime: Python 3.12.3 and websockets 16.1.1.
- Version: tested final-aware export f115f048f59d (SHA-256 source manifest); historical T3 baseline A acef9a3.
- Execution: unchanged public harness, source seed 3103, sequential runs and a fresh server for each repetition.
- Aggregation: T1–T5 each use three runs per configuration and report medians of per-run metrics. T3 whiskers show observed min/max. The separate 300 s RSS observation is a single supplementary run.
- Network: source, server and simulated clients run on same-host loopback. T1–T4 and RSS use no impairment; T5's isolated namespace and delay/loss profile are described in Section 4.6.

Metrics: Caption Age is client receive time minus source_ts; P50/P95/P99 and Fresh Caption Ratio (T=500 ms) use accepted live updates. Final Caption Completeness divides unique received/recovered finals by generated finals; Fresh Final Caption Ratio uses generated finals as denominator. Normal Update Coverage divides distinct received live revisions by generated revisions. Public HISTORY-response time ends at HISTORY receipt; oracle Recovery time ends only when all required finals and active state arrive (Appendix B).

Table 1. T2/T4/T5 medians of three runs. Fresh means age ≤500 ms; final completeness includes recovery. All runs recorded zero duplicate/stale captions and connection errors.

Test	P50 (ms)	P95 (ms)	P99 (ms)	Live fresh (%)	Fresh finals (%)	Final complete (%)	Normal Update Coverage (%)
T2	3.10	4.17	4.44	100.00	100.00	**100.00**	100.00
T4	3.94	5.95	6.75	100.00	98.42	**100.00**	98.25
T5	256.97	703.92	978.55	87.36	86.94	**100.00**	99.98

4.2 T1: Basic Correctness

- Workload: 20 clients × 20 s × 3 runs; 22 final segments per run.

Results: P50/P95/P99 = 1.34/1.97/2.25 ms; Fresh Caption Ratio, Final Caption Completeness and Normal Update Coverage were **100.00%**. All runs recorded zero duplicate/stale captions and connection errors. This validates the basic public workload.

4.3 T2: Public Scale

- Workload: 100 clients × 40 s × 3 runs; 49 final segments per run.

Table 1 summarizes the public-scale results. All source revisions and finals reached all 100 simulated clients; the results apply to this loopback workload.

4.4 T3: Slow-Client Isolation and Before/After Comparison

- Versions: Baseline A vs Current (Section 4.1).
- Change tested: the overload policy with final-aware queue , and the slow-reader drop with keepalive shortened from 20 s/20 s to 2 s/5 s .
- Workload: 100 clients × 60 s × 3 runs per version; 10 slow readers wait 500 ms per receive; 76 final segments/228 revisions per run.
- Groups: 90 normal and 10 slow clients; results are reported separately.

Table 2. Accepted live-caption latency and freshness; unique final completeness includes HISTORY recovery. Fresh means age ≤500 ms. Normal Update Coverage counts received revisions, not recovered finals.

Version / group	P50 (ms)	P95 (ms)	P99 (ms)	Fresh (%)	Final (%)	Coverage (%)
Before / normal	2.93	5.52	6.00	100.00	100.00	100.00
Before / slow	14,712.05	28,076.50	29,127.81	1.69	51.32	51.75
Current / normal	3.04	5.66	6.36	100.00	100.00	100.00
Current / slow	6,954.39	13,355.78	13,806.27	2.65	**100.00**	49.56

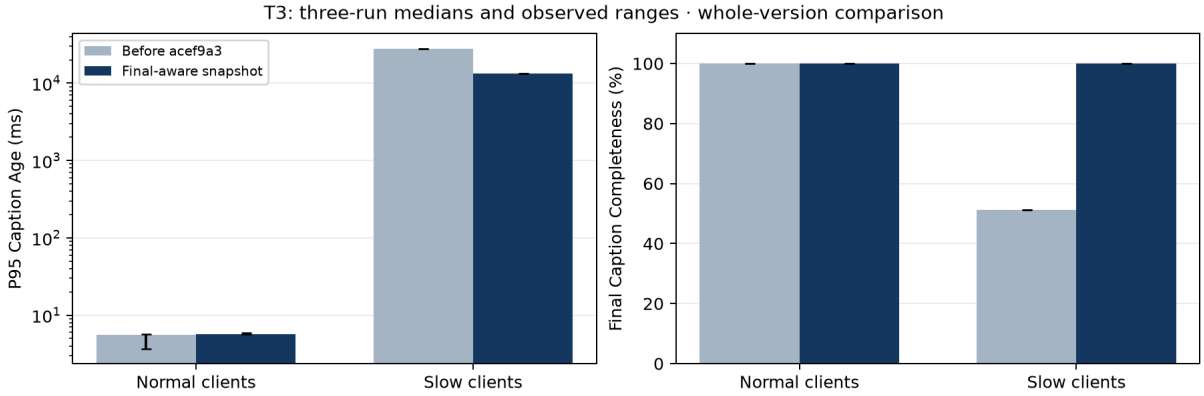


Figure 3. Normal/slow T3 latency and final completeness before and after the full version change. Latency uses a log scale and excludes HISTORY items; the accepted-live sample populations differ between versions.

Normal clients retained **100%** Normal Update Coverage and Final Caption Completeness in both versions. Slow-client P95 decreased from 28.08 s to 13.36 s; final completeness increased from 51.32% to **100.00%**. Current runs still recorded 20 CONNECT_ERROR events from keepalive closes and 380 recovered items per run (medians), with zero duplicate/stale captions. Slow-client Fresh Final Caption Ratio remained 1.32%: eventual completeness did not provide timely delivery. Median HISTORY-response P95 was 2.62 ms, not complete recovery time. This compares whole default versions; Appendix C separately examines protected scheduling.

The result illustrates the intended overload trade-off. The current design does not attempt to preserve every provisional revision for a receiver that cannot consume them in real time; instead, it preserves finalized state and the latest useful active state while allowing superseded provisional work to be removed. Consequently, slow-client Normal Update Coverage remains lower than for healthy clients, but final completeness improves to 100%, and the accumulated delivery backlog is substantially reduced. Normal clients remain unaffected, retaining 100% coverage and final completeness.

Slow readers whose reply to a keepalive ping is delayed by more than 5 s are closed with code 1011 and catch up through HISTORY. Because about 8 s of backlog sits in the client library's receive queue before pings are delayed, slow readers can still be over 10 s behind when dropped (P95 13.4 s).

4.5 T4: Reconnect and Recovery

- Workload: 100 clients × 60 s × 3 runs; 20% disconnect at 15 s for 5 s; 76 finals per run.

T4 recovered 140 HISTORY items through 20 reconnects; connection-to-HISTORY P95 was 3.46 ms. This is not full recovery time. The separate oracle checked all required finals and active state: large-history median 17.03 ms (Appendix B). Offline provisionals explain lower coverage in Table 1.

4.6 T5: Network Impairment

- Network: isolated Linux network namespace; supplied delay_loss.sh on lo (120 ms ±30 ms normal jitter, 3% loss). The source-to-server and server-to-client paths both traverse this interface; these are not end-to-end one-way delay values. Profile reset after all runs.
- Workload: 100 clients × 40 s × 3 runs; 49 finals per run.

All finals arrived, but not all live updates met the 500 ms target (Table 1). This is consistent with TCP retransmission, ordered delivery and jitter; without packet traces, their individual effects are not isolated. Appendix A shows the receive-age distributions.

4.7 Server Memory Observations

Supplementary RSS measurements found T3 server peaks of 33.54–33.63 MiB. In one 300 s workload, RSS rose from an early median of 32.35 MiB to a late median of 36.63 MiB, with peak 36.66 MiB. These observations do not establish a memory plateau or identify the cause of growth. Methods and the detailed figure are in Appendix D.

5. Failures, Trade-offs and Limitations

Failures and fixes

- **Slow readers lost half their finals**: In Baseline A, slow readers received only 51% of finals in T3. Their backlog sat in client-side buffers, where the server could neither see it nor clean it up. Fix: the 2 s / 5 s keepalive drop with HISTORY recovery (Section 4.4), which raised completeness to 100%.
- **Large recoveries were rejected**: The earlier recovery path sent all missed finals in a single HISTORY message through the live queue. When the missed context was large, or the queue was small, the whole recovery was rejected before anything was sent: 0 of 3 runs succeeded in four of the five Appendix B cases. Fix: recovery is now streamed in HISTORY batches of at most 1 MiB, ahead of live delivery. All 15 runs then returned every required final and draft, with no duplicates.

Trade-offs

- **Coverage vs completeness**: Under overload, superseded drafts are dropped so that finals and the newest draft arrive sooner. Lower revision coverage for slow clients is the tradeoff.
- **Keepalive timeout**. A short timeout recovers slow readers sooner, but on a lossy or congested link it could drop healthy clients. A long timeout leaves slow readers behind for longer. With 5 s, T5 (120 ms delay, 3% loss) recorded no connection errors, but this might change under harsher conditions.
- **Reconnects instead of a smarter queue**: Dropping a slow reader costs a reconnect and a HISTORY transfer, but it is the only way to discard a backlog that has already left the server.

Limitations

- **Loss of older captions**: A client offline longer than its requested history_seconds (at most 60 s) does not recover older finals.
- **Lane exhaustion**: If either queue lane fills completely, the client is still closed and its pending data discarded; it must recover by reconnecting.
- **No persistence or authentication:** server crash loses all previous captions and clients' hello are not authenticated

6. Future Work and FYP Direction

Fresher slow readers: A client already dropped multiple times for lag could be served in DEGRADED (or finals-only) mode from the moment it reconnects, instead of falling behind again before the next drop.

Making the keepalive timeout adaptive: shorter for clients with a history of lag and longer on lossy links, it would recover slow readers sooner without dropping healthy clients.

Profile memory: Record queue, history and task counts alongside RSS during longer runs.

Authentication and transport security: Require a token in SOURCE_HELLO and CLIENT_HELLO so only an authorized source can publish a lecture.

More robust input handling: Treat every source frame defensively: catch decoding and parsing errors of any kind.

FYP direction: receiver-aware captions over QUIC. WebSocket runs over one TCP stream, so a single lost packet delays every caption behind it (Section 4.6). A final-year project could carry captions over QUIC (WebTransport), with a separate stream per segment or unreliable datagrams for drafts, so a loss delays only its own segment.

References

[1] CS3103 Assignment 4: LiveRecall handout, updated 17 October 2026, sections 8, 10 and 12.

[2] Repository snapshots acef9a3 and 18f0dbf; tested final-aware export f115f048f59d with source hashes. Raw results/scripts: experiments/delivery_20261103/ and experiments/final_aware_delivery/.

Appendix A. Additional Experimental Figures

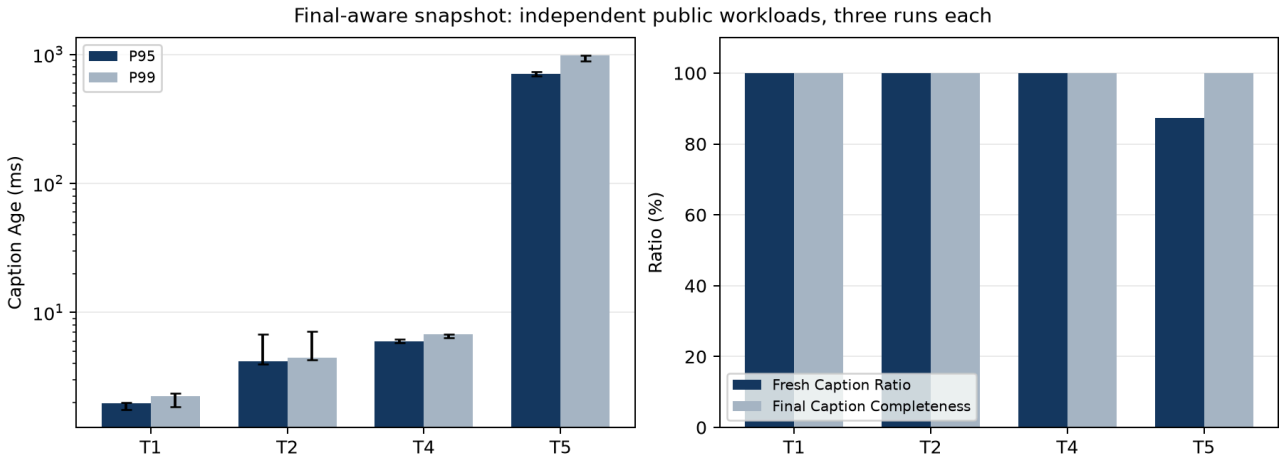


Figure 4. T1/T2/T4/T5 medians of three per-run metrics; whiskers show observed min/max, not confidence intervals. Latency excludes HISTORY; finals include recovery. T3 is shown separately in Figure 3. Workloads differ, so this is not a controlled scaling curve.

Unshaped T1/T2/T4 retain **100%** live freshness, while T5 delay/loss increases tail age and lowers freshness despite **100%** final completeness. T4's offline period reduces fresh-final ratio because recovered finals are older than the 500 ms target.

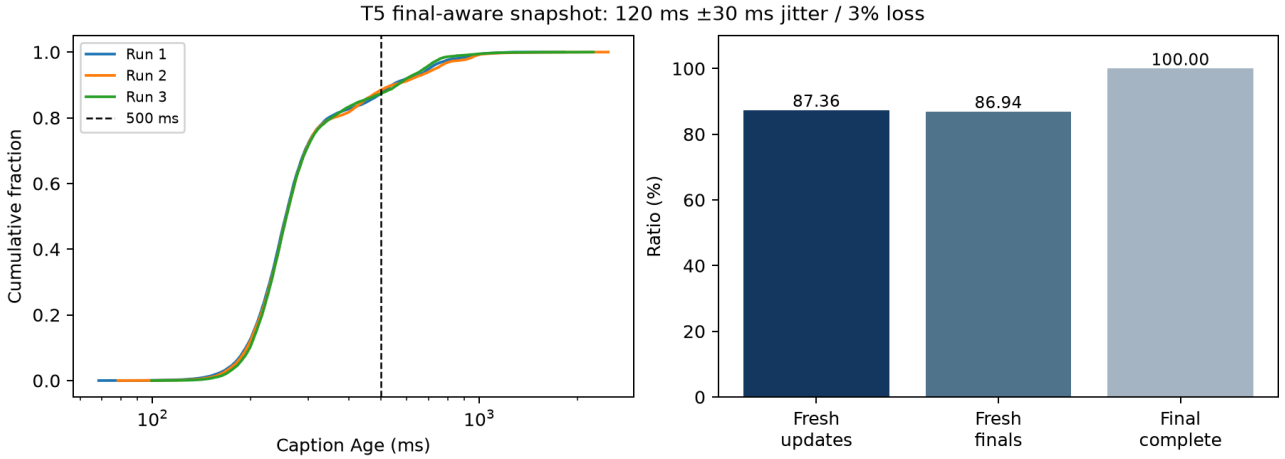


Figure 5. T5 per-run empirical cumulative live-age distributions and freshness/final completeness, using the accepted three-run batch. Each curve includes all accepted live updates from one run; the dashed line marks 500 ms. Latency uses a log scale.

All three runs converge to **100%** Final Caption Completeness, while about 87% of live updates arrive within 500 ms. Receive-age tails show that eventual reliability and timely delivery are separate properties. The fixed source seed does not fix netem's random loss/jitter.

Appendix B. Complete Recovery Verification

Method: GB10 loopback, Python 3.12.3, no network impairment; five cases × three fresh connections per version. Before is Baseline B(18f0dbf); current batched recovery is final-aware export f115f048f59d (recorded source hashes). The independent oracle checks every required final and latest active payload, duplicates and ordering. Completion runs from successful connection until all required state arrives; first-HISTORY response is recorded separately.

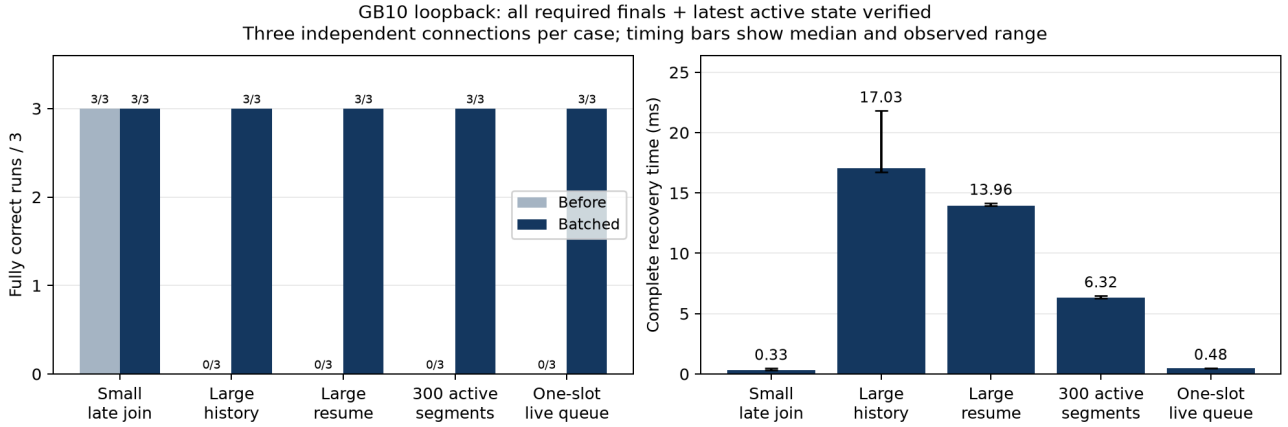


Figure 6. Oracle-verified complete recovery. Bars show successful runs out of three and batched completion medians with observed min/max. Failed pre-fix runs have no completion time, rather than zero latency.

- Small late join: 20 finals + 3 active; both versions 3/3 correct; current median 0.33 ms.
- Large history: 96 finals of approximately 20 KB each + 3 active; before 0/3, current 3/3; 17.03 ms, two HISTORY batches.
- Large resume: 16 known finals excluded; before 0/3, current 3/3; 13.96 ms, two batches.
- Many active: 10 finals + 300 active, exceeding the 256-slot live queue; before 0/3, current 3/3; 6.32 ms.
- One-slot live queue: 10 finals + 16 active; before 0/3, current 3/3; 0.48 ms.

The pre-fix implementation rejected larger/capacity-constrained contexts before delivery. Every current run returned all expected state with zero duplicates. This validates the tested recovery boundary, not arbitrary congestion or hidden workloads. Expired/count-pruned history, individually oversized messages and live-lane exhaustion remain limits. Methods, source hashes and per-run results: experiments/final_aware_delivery/.

Appendix C. Final-Aware Scheduling and History Capacity

Method: production queue/policy code under virtual time; five cases × three source seeds × two implementations. Service is 100 messages/s. Normal load is 5 finals/s plus 10 provisionals/s; overload cases use 90 or 99 finals/s plus 40 provisionals/s. Source duration is 20 s with a 2 s drain. FIFO uses 32 slots/64 KiB (two slots in the full-final case). Final-aware storage adds a separate 1,000-segment/64 KiB active map. This compares storage and scheduling together, not priority alone; it does not model WebSocket/TCP, reconnects or network Caption Age.

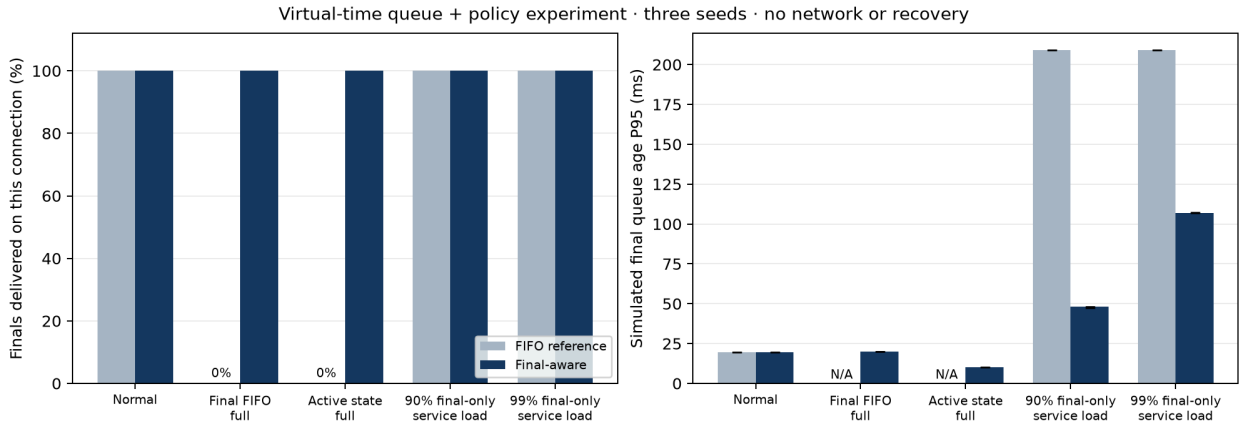


Figure 7. Controlled scheduler outcomes, medians of three seeds. Failed FIFO admissions model eviction and discard the pending connection backlog; reconnect recovery is not simulated. Zero completeness is labeled explicitly, not an absent result.

All 15 final-aware cases delivered every final and retained all required latest active state, without eviction. Normal Update Coverage stayed **100%** under normal load. At 90%/99% final-only service load, median final queue-age P95 fell from 209.13/208.97 ms to 48.02/107.07 ms. The backlog override avoids a fixed 80% final-service ceiling; these are simulated queue ages, not measured networking latency.

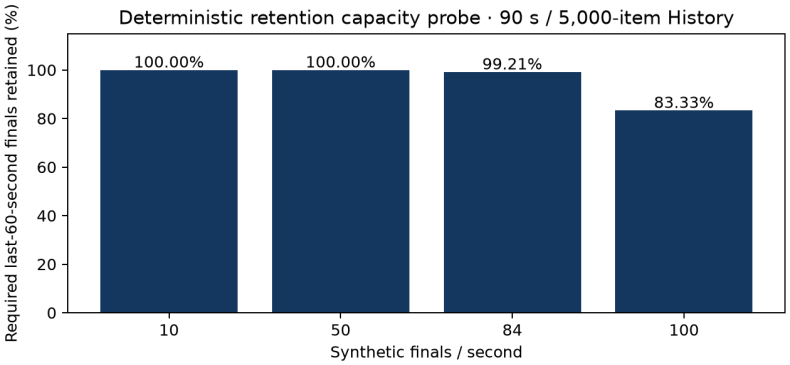


Figure 8. Separate History capacity probe: inject finals across 90 virtual seconds, then query the last 60 s. The unchanged 5,000-item cap returns 600/600, 3,000/3,000, 5,000/5,040 and 5,000/6,000 required finals at 10/50/84/100 finals/s. A time window alone does not guarantee 60 s of recoverable content. These rates are diagnostic, not claims about hidden workloads.

Appendix D. Supplementary Server Memory Observation

Method/metrics: server-child VmRSS sampled every 0.5 s, tracked by PID; compare three T3 runs/version and one additional 300 s current run with the same 90/10 client mix. Report peak, early/late median and post-60 s slope.

An external observer sampled server-child VmRSS every 0.5 s. Peak T3 RSS ranged from 31.93–31.99 MiB before to 33.54–33.63 MiB for the final-aware export (three runs each). A separate 300 s workload produced a 308.9 s trace including startup/teardown: peak 36.66 MiB, median 32.35 MiB during seconds 10–30 and 36.63 MiB in the final 30 s. The post-60 s slope was about 0.72 MiB/min.

All application structures have fixed limits (Section 3.1); the observed growth was not attributed to them.

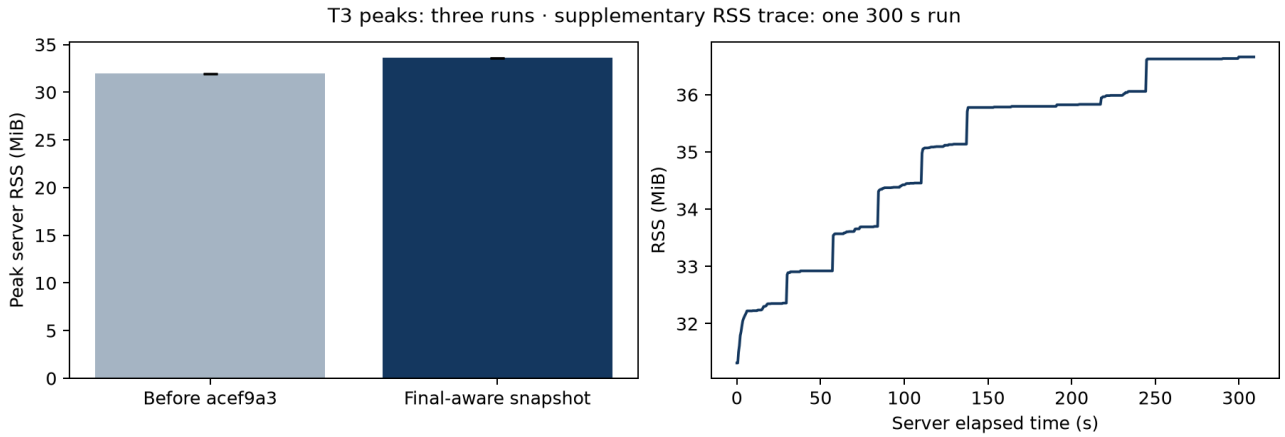


Figure 9. Peak server RSS across three T3 runs (median and min/max), and the final-aware export's supplementary 300 s workload trace. Sampling includes startup/teardown; RSS excludes clients and kernel socket buffers. One long run provides no confidence interval.